Generic Iteration Workflow

Chapter 12

Iterative and Incremental Development

- Part III of the text revisits our earlier discussion in iterative and incremental development
  - now, we know more about the core workflows
- We revisit how the workflows are combined for each of the phases
  - inception, elaboration, construction, transition
- Finally we look at tailoring the UP to make it work better
- First look at things common to all phases

General Iteration Workflow

- The common pattern behind all four phases
- Includes core workflows
  - requirements, analysis, design, implementation, test
- Includes planning and assessment
- Plans based largely on risk management
- Resources also need to be estimated

Need for Balance

- Many activities need to happen to develop software
  - code functions, lay out forms, architect, talk to users, mitigate risks, plan & estimate, etc.
- Each iteration requires balancing the activities
  - time and effort limit what can be done
  - do the most important things first
- Failure to balance the activities leads to crisis

What’s Important?

- Early iterations
  - critical risks
  - key use cases
  - architectural issues
  - development environment
- Later iterations
  - development-oriented activities
  - implementation and testing
  - resolving performance issues

Phases: Highest Work Decomposition

- Phases follow one after another
  - Nonoverlapping
- Inception, elaboration, construction, transition
- Each phase has one or more iterations

Inception
Establishes Feasibility

- Establish the business case
- Develop the use cases to support this
- Scope the system
  - define system boundary and interfaces
- Outline candidate architecture - make believable
- Identify critical risks
- Demonstrate system will meet user needs
  - proof of concept prototype
- Feasibility established, move to elaboration

Elaboration - Can We Do It?

- Goals
  - Produce the stable architecture
  - Produce a stable cost estimate
- Activities
  - create architectural baseline including executable
  - identify significant risks
  - specify quality levels (e.g. reliability, response times)
  - capture most use cases (80% of functionality)
  - estimate schedule, staff, cost

Construction - Build it

- Build initial operational capability
- Most staff, longest schedule of phases
- Poor requirements & design gives overruns here
  - temptation to hack out the system, taking even longer
- Finish use case capture and detailing
- Finish analysis, design, implementation and test
- Maintain integrity of architecture
- Monitor and mitigate significant risks

Transition - Give It to the Users

- Refine the system in operations
- May involve site preparation
- Finish manuals and product release documents
- Tune software for operational use
- Correct defects found from beta tests
- Modify software to correct surprise problems
- Ends in product release
  - project should document lessons learned

Generic Iteration Revisited

- Each iteration includes core workflows
  - requirements, analysis, design, implementation, test
  - plus planning and assessment
- Workers specialize in the activities required
  - see figure 12-3 in text
  - use cases start first
  - analysts cut across most workflows
  - test work flow should start early

Workers and Workflows

	System Analyst	Use-case Specifier	User-Interface Designer	Architect	Use-Case Engineer	Component Engineer	Test Engineer	System Integrator	Integration Tester	System Tester
Requirements	*	*	*	*						
Analysis				*	*	*				
Design				*	*	*				
Implementation				*		*		*		
Test						*	*	*	*	*

Planning Precedes Doing

- At start of project know the least
- Still need to plan the work at highest level
- You know the number of phases
- You know the information that led to initiating the project
- You know about the domain and similar projects you have worked on
- As you get more knowledge, plans can become more detailed

Plan the Four Phases

- Allocate schedule times for each phase
  - tentative until end of elaboration
- Schedule major milestones
- Estimate how many iterations per phase and what they will do
- Put it all into a project plan

- Expect the first iteration to be difficult

Plan Iterations

- Estimate time to spend on iteration
- Estimate content to put in iteration
  - which use cases to do (at least partially)
  - which technical risks to eliminate
  - any changes to requirements or rework required
  - which subsystems or components to implement (partially or completely)
- Document in iteration plan

Plan Evaluation Criteria

- Outline key objectives for each iteration
  - resolving requirements ambiguities, for example
  - should be measurable
- Establish minimum completion criteria for each iteration
  - otherwise it will go on forever
  - functional requirements in terms of use cases
  - non-functional requirements linked to use cases
  - other non-functional requirements

Risks Affect Project Planning

- Manage a risk list
  - review it weekly
- Risks affect the iteration plans
- Schedule actions to deal with the risks
- Some details on following slides

Risks

- Risk: potential problem that could adversely impact
  - planned schedule, budget, or success of the project
- Not just something difficult
- Should have some potential for occurrence
- If it occurs, it will generally alter the normal plan

Risk Management

- Risks must be identified and then managed:
  - preventing the risk occurrence
    - selecting hardware that is available from multiple sources.
  - monitoring the risk
    - auditing the progress of contractors on whom you have a dependency.
  - taking corrective action
    - use a simulator until the hardware is ready, or hire consultants to make up for your staff shortfall.

Elements of Risk Management

- Risk Assessment
  - risk identification
  - risk analysis
  - risk prioritization
- Risk Control
  - risk-management planning
  - risk resolution
  - risk monitoring

Risk Identification

- Identify factors that pose a risk
- Common risks
  - making one of the “Classic Mistakes”
  - ignoring development basics
  - failing to manage risks
- Rapid Development, Steve McConnell, has a list of schedule risks
  - good to review them against any project

Most Common Schedule Risks

- Feature creep
- Requirements or developer gold plating
- Shortchanged quality
- Overly optimistic schedules
- Inadequate design
- Silver-bullet syndrome
- Research-oriented development
- Weak personnel

Risk Analysis

- Analyze each risk for impact
  - risk exposure is probability of loss times size of loss = expected loss
  - Works for both schedule (weeks) and cost ($)

Risk	*Probability of loss*	*Size of loss (weeks)*	*Exposure (Weeks)*
Optimistic schedule	50%	5	2.5
Additional features from marketing	35%	8	2.8
Unstable graphics interface	25%	4	1.0

Risk Prioritization

- Which risks to attend to?
  - 80% of time fixing 20% of problems
  - fix the right ones!
- Sort by exposure

Risk	*Probability of loss*	*Size of loss (weeks)*	*Exposure (Weeks)*
Additional features from marketing	35%	8	2.8
Optimistic schedule	50%	5	2.5
Unstable graphics interface	25%	4	1.0

Risk Management Planning

- Go through each risk identified
  - how can the risk be prevented?
  - how will the risk be monitored?
  - how will the risk be resolved if it occurs?
- How will new risks be identified?
- All of the above documented in a risk management plan

Risk Resolution

- Depends on specific risk - use creativity
  - avoid the risk
  - move the risk off the critical path
  - invest in finding out more about the risk
  - eliminate the root cause
  - build the risk damage into the schedule
  - publicize the risk
  - control the risk
  - remember the risk on later projects

Risk Monitoring

- Keep top-10 risks list
- Review every week
- include risk resolution progress for each

- Risks that are watched are easier to manage

Use-Case Prioritization

- Risks can be used to prioritize use cases covered in an iteration
- Early phases, concentrate on risks related to scope and architecture
- Later phases concentrate on adding functionality
- After risks covered, add use cases in customer priority

Resources Needed

- Typical project profile:
  - inception: 10% schedule, 5%effort
  - elaboration: 30%schedule, 20% effort
  - construction: 50% schedule, 65% effort
  - transition: 10% schedule, 10% effort
- Estimate based on similar projects, past experience, or costing models
  - usually need an estimate of size

Hazards to Avoid in Estimating

- Estimating only a subset of activities
- Underestimating the size and complexity of the system
- Overestimating staff skills
- Overestimating the (positive) impact of new tools and techniques
- Underestimating the learning curve
- Trading quality and completeness for schedule

Estimation Process Overview

- Estimate the size of the project (lines of code or function points)
- Estimate the effort (man-months)
- Estimate the schedule (calendar months)
- Provide estimates in ranges and refine the ranges as project progresses

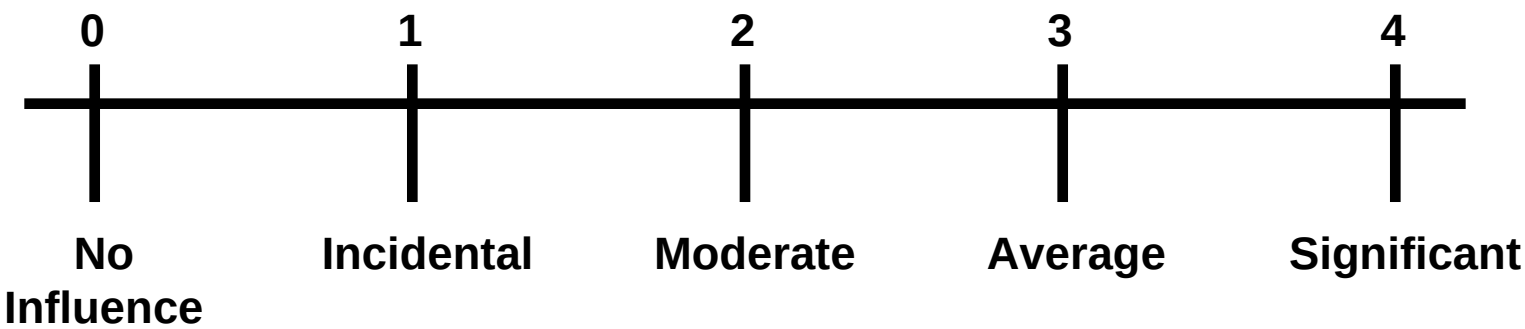
Size Estimation Techniques

- Algorithmic models (function points)
- Expert judgment
- Analogy
- Bottom-up
- Delphi model with experts

Function Point Analysis

- Due to A. J. Albrecht of IBM
- Measures size of software without counting lines of code
- Can be counted from requirements descriptions
  - language independent
  - related to the volume of functionality
  - easier to estimate from requirements descriptions.
- Best documented in J.B. Dreger, *Function Point Analysis*

Function Points - Fi Values



1. Does the system require reliable backup?
2. Are data communications required?
3. Are there distributed processing functions?
4. Is performance critical?
5. Will the system run in an existing environment?
6. Does the system require on-line data entry?
7. Does the on-line data entry require the input transaction to be built over multiple screens or operations?
8. Are the master files updated on-line?
9. Are the inputs, outputs, files, or inquiries complex?
10. Is the internal processing complex?
11. Is the code designed to be reusable?
12. Are conversion and installation included in design?
13. Is the system designed for multiple installations in different organizations?
14. Is the application designed to facilitate change and ease of use?

Function Points Computation

Measurement Parameter	Count	Weighting Factor Simple	Average	Complex		Product
Number of user inputs		x 3	4	6	=	
Number of user outputs		x 4	5	7	=	
Number of user inquiries		x 3	4	6	=	
Number of files		x 7	10	15	=	
Number of external interfaces		x 5	7	10	=	
Count - Total						

$$FP = \text{count-total} * (0.65 + 0.01\Sigma F_i)$$

Relationship of LOC to FP

- Approximate number of lines of code for one function point are:

Language	*LOC/FP*
Assembly	300
C	128
COBOL	100
FORTRAN	100
Pascal	90
Ada 83	70
C++, Java	53
Ada 95	49
VB, Delphi	29
Fourth Generation Languages	20
Automatic Code Generators	15

Assess Iterations and Phases

- For iterative methods to work, need to assess what has been accomplished
  - at end of each iteration
  - at end of each phase
- Project Manager primarily responsible
- Two objectives
  - replan next iteration based on current results
  - modify process, tools, training, etc., based on experience

What Was Achieved?

- Examine accomplishments with respect to evaluation criteria and the plan
- Is work on budget and schedule?
- Is work meeting quality requirements?
  - Based on tests or observations by concerned stakeholders

Things Not Achieved - Oops

- Often iterations do not achieve all criteria
  - if everything always perfect, no need for iterations
- Modify future plans based on current realities
  - modify or extend use-case model
  - modify or extend architecture
  - modify or extend developed subsystems
  - search for further risks
  - add skills or training to team
- Modify budget or schedule

Going Into the Next Iteration

- Determine if ready to proceed to next iteration
- Decide where any rework will be scheduled
- Plan the next Iteration in detail
- Update the plan for subsequent iterations
- Update risks and project plan
- Compare actual cost and schedule against those planned
  - may result in further modifications